OSS CAPSTONE AUDIT — Git

(Version Control System)

Student name: DIXITA NEMA

Registration Number: 24BAI10802

Course: Open Source Software (NGMC)

Institution: VITyarthi

Date: 31-03-26

Slot: A24

Software Chosen: Git

Abstract

This project presents a comprehensive audit of Git, a distributed version control system that has become a cornerstone of modern software development practices. The purpose of this audit is to analyze Git from both a technical and open-source perspective, focusing on its origin, design philosophy, licensing model, and impact on the global developer community. Git enables developers to track changes efficiently, collaborate across distributed teams, and maintain a complete history of their work. In addition to the theoretical analysis, this project also includes the implementation of five Bash shell scripts that demonstrate practical Linux command-line skills. These scripts are designed to reflect real-world system operations such as system monitoring, package inspection, log analysis, and file auditing. The project emphasizes the importance of open-source software in promoting transparency, collaboration, and innovation. It also highlights how tools like Git empower developers to build scalable and maintainable software systems. Through this project, both conceptual understanding and hands-on technical skills are developed. The integration of theory and practice ensures a holistic learning experience. Overall, this audit showcases the relevance of Git in the open-source ecosystem and demonstrates the practical application of Linux-based scripting in real-world scenarios.

Introduction to Open Source Software

Open Source Software (OSS) refers to software whose source code is made freely available to the public for use, modification, and distribution. Key Principles:

- Free redistribution
- Access to source code
- Derived works allowed
- Integrity of author's code

OSS has revolutionized software development by enabling global collaboration and reducing dependency on proprietary systems.

Unlike proprietary software, OSS encourages collaborative development and allows developers from around the world to contribute to the improvement of the software. The concept of open source is built on principles such as transparency, community participation, and shared knowledge. One of the key advantages of OSS is that it eliminates dependency on a single vendor, thereby increasing flexibility and reducing costs. Developers can study the code, identify bugs, and enhance functionality without restrictions. OSS also promotes rapid innovation because multiple contributors can work simultaneously on different aspects of the software. Popular examples of OSS include the Linux operating system, Git, Apache web server, and Mozilla Firefox. Open-source licensing plays a crucial role in defining how software can be used and distributed. These licenses ensure that the software remains free and accessible to everyone. OSS has significantly influenced the software industry by fostering a culture of collaboration and continuous improvement. It is widely used in

education, research, and enterprise environments. Governments and organizations also prefer OSS for its security and transparency. Overall, open-source software represents a powerful model for building robust, scalable, and community-driven technologies.

Introduction to Git

Git is a distributed version control system designed to track changes in source code during software development. Key Features:

- Distributed architecture
- Fast performance
- Secure (SHA-1 hashing)
- Branching and merging
- Open-source and free
  Git allows developers to work independently and merge changes efficiently.

It allows multiple developers to work on the same project simultaneously without interfering with each other's work. One of the defining features of Git is its distributed architecture, where every user has a complete copy of the repository, including its history. This ensures that no single point of failure exists, making Git highly reliable. Git provides powerful branching and merging capabilities, enabling developers to experiment with new features without affecting the main codebase. It also maintains a detailed history of changes, allowing users to revert to previous versions if necessary. Git uses a snapshot-based approach rather than storing differences, which improves performance and efficiency. Security is another important aspect, as Git uses cryptographic hashing (SHA-1) to ensure data integrity. Git is widely used in conjunction with platforms like GitHub and GitLab, which provide additional collaboration tools. It supports both small-scale and large-scale projects, making it versatile and scalable. Developers can work offline and sync changes later, which enhances productivity. Git has become an essential tool in modern DevOps practices and continuous integration workflows. Its flexibility, speed, and robustness make it the most popular version control system in the world today.

History and Origin of Git

Git was created in 2005 by Linus Torvalds, who is also the creator of the Linux kernel. Background:

- Linux developers were using BitKeeper
- BitKeeper revoked its free license
- Linus Torvalds developed Git as a replacement

Git quickly became the standard version control system due to its speed and flexibility.

The development of Git was triggered by a licensing issue with BitKeeper, a proprietary version control system that was previously used for Linux kernel development. When BitKeeper revoked its free license, the Linux community needed an alternative solution that was both efficient and open-source. Linus Torvalds decided to develop Git with a focus on speed, simplicity, and distributed collaboration. Within a few weeks, the initial version of Git was released, demonstrating remarkable performance and functionality. Git was designed to handle large projects like the Linux kernel, which involves contributions from thousands of developers worldwide. Junio C Hamano later became the maintainer of Git and played a significant role in its development and adoption. Over the years, Git has evolved into a powerful tool with widespread usage across industries. It has become the standard version control system for open-source and commercial projects alike. The success of Git can be attributed to its efficient design and strong community support. Today, millions of developers rely on Git for managing their code. Its origin story highlights the importance of open-source solutions in overcoming limitations imposed by proprietary software. Git continues to evolve with new features and improvements, maintaining its relevance in modern software development.

License Analysis — GPL v2

Git is licensed under the GNU General Public License version 2 (GPL v2), which is one of the most widely used open-source licenses. Key Features of GPL v2:

- Freedom to use software
- Freedom to modify
- Freedom to distribute
- Copyleft (derivatives must also be open-source)

This ensures that Git remains free and open for all users.

The GPL v2 ensures that the software remains free and open for all users. It grants users the freedom to run, study, modify, and distribute the software without restrictions. One of the key features of GPL v2 is its “copyleft” provision, which requires that any modified versions of the software must also be distributed under the same license. This ensures that the software and its derivatives remain open-source. The license promotes collaboration by allowing developers to build upon existing code. It also protects the rights of users by preventing the software from being turned into proprietary software. GPL v2 has been used by many major projects, including the Linux kernel. It encourages transparency and accountability in software development. Developers are required to provide access to the source code when distributing the software. This fosters trust and reliability among users. The license also includes disclaimers that limit liability for the developers. Overall, GPL v2 plays a crucial role in maintaining the open-source nature of Git. It ensures that Git remains a freely accessible tool for developers worldwide. The license aligns with the core principles of open-source software, promoting innovation and collaboration.

Importance of Git in Open Source Ecosystem

Git plays a central role in the open-source ecosystem and has become one of the most essential tools for modern software development. It enables collaborative development by allowing multiple developers to work on the same project simultaneously without interfering with each other's changes. Through its distributed architecture, every contributor has a complete copy of the repository, which increases reliability and eliminates dependency on a central server. Git also provides efficient version tracking, allowing developers to maintain a detailed history of changes, revert to previous versions, and identify who made specific modifications.

One of the major reasons for Git's popularity is its seamless integration with code hosting platforms such as GitHub, which provide additional features like pull requests, issue tracking, and project management tools. These platforms make it easier for developers across the globe to collaborate on open-source projects. Git also plays a crucial role in continuous integration and continuous deployment (CI/CD) workflows, where code changes are automatically tested and deployed, improving software quality and reducing development time.

Furthermore, Git encourages experimentation through branching and merging, allowing developers to create separate branches for new features without affecting the main codebase. This promotes innovation and safer development practices. Git is used by millions of developers worldwide and supports projects of all sizes, from small personal repositories to large-scale enterprise systems. Its speed, flexibility, and strong community support make it the backbone of the open-source ecosystem. Overall, Git has transformed the way software is developed, shared, and maintained, making it an indispensable tool in today's technology landscape.

FOSS Ecosystem

Git is a fundamental component of the open-source world.

Dependencies

- Written in: C
- Uses System Libraries

Ecosystem Impact

- GitHub, GitLab, and similar platforms are built on top of Git
- Facilitates collaboration among developers

Community

- Maintained by developers worldwide
- Allows contributions through mailing lists and repositories role

Git is crucial for:

- Version Control

- Collaboration
- Open-source Development

Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative
Cost	Free	Paid
Security (who can audit the code?)	Transparent	Closed
Support and reliability	Community	Company
Freedom to modify	High	Limited
Community vs corporate control	User-Controlled	Vendor-Controlled
Your overall verdict		

Linux Environment and Bash Scripting

Linux is a widely used open-source operating system that provides a powerful and flexible environment for software development and system administration. One of its key strengths is the command-line interface (CLI), which allows users to interact directly with the system using text-based commands. This interface is highly efficient and provides greater control compared to graphical user interfaces. Linux is known for its stability, security, and performance, making it a preferred choice for developers, system administrators, and organizations worldwide.

Bash (Bourne Again Shell) is the default shell in most Linux distributions and serves as a command interpreter. It allows users to execute commands, manage files, and automate repetitive tasks through scripting. Bash scripting is a powerful tool that enables users to write scripts to perform complex operations, such as system monitoring, file manipulation, and process management. It supports features like variables, loops, conditional statements, and input/output redirection, making it suitable for both simple and advanced tasks.

In this project, Bash scripting is used to demonstrate practical system-level operations. The scripts developed showcase various aspects of Linux usage, including system information retrieval, package inspection, disk and permission auditing, log file analysis, and interactive user input handling. These scripts highlight the efficiency and versatility of the Linux environment. By using Bash, repetitive tasks can be automated, reducing manual effort and increasing productivity. Additionally, Bash scripts are lightweight, portable, and easy to execute on any Linux system.

Overall, the Linux environment combined with Bash scripting provides a strong foundation for understanding system-level programming and open-source tools. It enhances problem-solving skills and prepares users for real-world scenarios in software development and system administration.

Script Implementation

Script 1: System Identity Report

This script displays essential information about the system, including the current user, hostname, operating system details, kernel version, system uptime, and hardware information such as CPU and memory. It helps in quickly understanding the system configuration and environment.

Code:

```
#!/bin/bash

# Script 1: Advanced System Identity Report


clear


echo "==============================================="

echo "       OPEN SOURCE SYSTEM REPORT"

echo "==============================================="


# Variables

STUDENT_NAME="Dixita nema"

SOFTWARE="Git"


# System Info

KERNEL=$(uname -r)

HOSTNAME=$(dixita)

USER_NAME=$(dixita-01-nema)

HOME_DIR=$HOME

UPTIME=$(uptime -p)

DATE=$(date)

ARCH=$(uname -m)


# Distro Info

if command -v lsb_release &>/dev/null; then

   DISTRO=$(lsb_release -d | cut -f2)

else

   DISTRO="Unknown Distribution"

fi


# CPU Info
```

```
CPU=$(grep -m1 'model name' /proc/cpuinfo | cut -d ':' -f2)

# Memory Info
MEM=$(free -h | awk '/Mem:/ {print $2}')

# Disk Info
DISK=$(df -h / | awk 'NR==2 {print $2}')

# Output
echo "Student Name   : $DIXITA_NEMA"
echo "Software       : $SOFTWARE"
echo "Hostname       : $HOSTNAME"
echo "Kernel Version : $KERNEL"
echo "Architecture   : $ARCH"
echo "Distribution   : $DISTRO"
echo "User           : $dixita-01-nema"
echo "Home Directory : $HOME_DIR"
echo "System Uptime  : $UPTIME"
echo "Date & Time    : $DATE"
echo "CPU            : $CPU"
echo "Memory         : $MEM"
echo "Disk Space     : $DISK"
echo "License        : GNU/Linux (GPL v2)"

echo "==============================================="
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

==========================================
        OPEN SOURCE SYSTEM REPORT
==========================================
./script1_system_identity.sh: line 17: dixita-01-nema: command not found
Student Name    :
Software        : Git
Hostname        : LAPTOP-L9HT61AT
Kernel Version  : 6.6.87.2-microsoft-standard-WSL2
Architecture    : x86_64
Distribution    : Ubuntu 24.04.4 LTS
User            :
Home Directory  : /home/dixita_nema
System Uptime   : up 1 hour, 1 minute
Date & Time     : Sun Mar 29 18:36:47 UTC 2026
CPU             :  AMD Ryzen 5 5500U with Radeon Graphics
Memory          : 7.4Gi
Disk Space      : 1007G
License         : GNU/Linux (GPL v2)
==========================================
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$
```

Script 2: Package Inspector

This script checks whether **Git** is installed on the system. If installed, it displays the version and additional package details using the system's package manager. It is useful for verifying software availability and system setup.

Code:

```
#!/bin/bash

# Script 2: Advanced Package Inspector (Git)


PACKAGE="git"


echo "======================================="

echo "   FOSS PACKAGE INSPECTOR"

echo "======================================="


# Check installation

if command -v $PACKAGE &>/dev/null; then

   echo "[✓] $PACKAGE is installed"

   VERSION=$($PACKAGE --version)

   echo "Version: $VERSION"


   echo "---------------------------------------"


   # Detect package manager
```

```
    if command -v dpkg &>/dev/null; then
        echo "Using dpkg (Debian-based system)"
        dpkg -s $PACKAGE | grep -E 'Version|Maintainer'
    elif command -v rpm &>/dev/null; then
        echo "Using rpm (RedHat-based system)"
        rpm -qi $PACKAGE | grep -E 'Version|License|Summary'
    else
        echo "Unknown package manager"
    fi
else
    echo "[✗] $PACKAGE is NOT installed"
fi

# Philosophy notes
echo "-------------------------------------"
echo "Open Source Philosophy Notes:"

case $PACKAGE in
    git) echo "Git: Decentralized collaboration tool enabling freedom in development";;
    apache) echo "Apache: Backbone of the open web";;
    mysql) echo "MySQL: Open-source database powering applications";;
    vlc) echo "VLC: Media freedom for everyone";;
    *) echo "General FOSS tool";;
esac

echo "======================================"
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

==============================================
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$ ./script2_package_inspector.sh
=====================================
   FOSS PACKAGE INSPECTOR
=====================================
[✓] git is installed
Version: git version 2.43.0
-------------------------------------
Using dpkg (Debian-based system)
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 1:2.43.0-1ubuntu7.3
Original-Maintainer: Jonathan Nieder <jrnieder@gmail.com>
-------------------------------------
Open Source Philosophy Notes:
Git: Decentralized collaboration tool enabling freedom in development
=====================================
```

Script 3: Disk and Permission Auditor

This script analyzes important directories in the Linux system and displays their permissions, ownership, and disk usage. It helps in monitoring storage usage and identifying potential permission-related issues.

Code:

```
#!/bin/bash

# Script 3: Advanced Disk & Permission Auditor


DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "======================================="

echo "   DISK & PERMISSION AUDIT REPORT"

echo "======================================="


for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld $DIR | awk '{print $1}')

    OWNER=$(ls -ld $DIR | awk '{print $3}')

    GROUP=$(ls -ld $DIR | awk '{print $4}')

    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)


    echo "Directory: $DIR"

    echo "Permissions: $PERMS"

    echo "Owner: $OWNER"

    echo "Group: $GROUP"
```

```
    echo "Size: $SIZE"
    echo "-------------------------------------"
  else
    echo "$DIR not found"
  fi
done

# Git Binary Check
echo "Checking Git Binary Location..."
which git

echo "======================================="
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$ ./script3_disk_permission_auditor.sh
=====================================
   DISK & PERMISSION AUDIT REPORT
=====================================
Directory: /etc
Permissions: drwxr-xr-x
Owner: root
Group: root
Size: 4.3M
-------------------------------------
Directory: /var/log
Permissions: drwxr-xr-x
Owner: root
Group: syslog
Size: 17M
-------------------------------------
Directory: /home
Permissions: drwxr-xr-x
Owner: root
Group: root
Size: 28K
-------------------------------------
Directory: /usr/bin
Permissions: drwxr-xr-x
Owner: root
Group: root
Size: 95M
-------------------------------------
```

```
-------------------------------------
Directory: /tmp
Permissions: drwxrwxrwt
Owner: root
Group: root
Size: 28K
-------------------------------------
Checking Git Binary Location...
/usr/bin/git
=====================================
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$
```

Script 4: Log File Analyzer

This script reads a specified log file and searches for a given keyword (default is “error”). It counts the number of occurrences and displays the most recent matching lines, helping in debugging and system monitoring.

Code:

```
#!/bin/bash

# Script 4: Advanced Log Analyzer


LOGFILE=$1

KEYWORD=${2:-error}

COUNT=0


if [ -z "$LOGFILE" ]; then

   echo "Usage: $0 <logfile> [keyword]"

   exit 1

fi


if [ ! -f "$LOGFILE" ]; then

   echo "File not found. Trying alternative logs..."


   for ALT in /var/log/syslog /var/log/messages; do

      if [ -f "$ALT" ]; then

         LOGFILE=$ALT

         echo "Using $LOGFILE"

         break

      fi

   done

fi


while IFS= read -r LINE; do

   if echo "$LINE" | grep -iq "$KEYWORD"; then

      COUNT=$((COUNT+1))

   fi

done < "$LOGFILE"


echo "-------------------------------------"
```

```
echo "Keyword '$KEYWORD' found $COUNT times"

echo "-------------------------------------"


echo "Last 5 matches:"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$ ./script4_log_analyzer.sh /var/log/sy
slog
-------------------------------------
Keyword 'error' found 8 times
-------------------------------------
Last 5 matches:
2026-03-29T17:35:32.225654+00:00 LAPTOP-L9HT61AT kernel: WSL (248) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-29T17:35:32.484599+00:00 LAPTOP-L9HT61AT snapd[203]: helpers.go:190: error trying to compare the snap system key: system-ke
y missing on disk
2026-03-29T17:35:33.992929+00:00 LAPTOP-L9HT61AT wsl-pro-service[207]: #033[33mWARNING#033[0m Could not ensure valid Landscape conf
iguration: could not ensure valid Landscape configuration: could not register distro to Landscape: could not enable Landscape: /usr
/bin/landscape-config: error: exit status 1.
2026-03-29T17:35:40.496922+00:00 LAPTOP-L9HT61AT kernel: WSL (2 - init-systemd(Ubuntu)) ERROR: WaitForBootProcess:3432: /sbin/init
failed to start within 10000ms
2026-03-29T17:46:57.533068+00:00 LAPTOP-L9HT61AT wsl-pro-service[207]: #033[33mWARNING#033[0m Exiting after could not ensure valid
Landscape configuration: could not register distro to Landscape: could not enable Landscape: /usr/bin/landscape-config: error: exit
 status 1.
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$
```

Script 5: Manifesto Generator

This script interacts with the user by taking input and generating a personalized open-source manifesto. The output is saved to a text file and displayed on the screen, demonstrating basic input handling and file operations in Bash.

Code:

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator


echo "======================================="

echo " OPEN SOURCE MANIFESTO GENERATOR"

echo "======================================="


# Input validation

while [ -z "$TOOL" ]; do

   read -p "1. Name an open-source tool: " TOOL

done


while [ -z "$FREEDOM" ]; do

   read -p "2. What does freedom mean to you? " FREEDOM
```

```
done

while [ -z "$BUILD" ]; do
   read -p "3. What will you build and share? " BUILD
done

DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"

echo "--------------------------------------"

echo "On $DATE, I believe in open source." > $OUTPUT
echo "I regularly use $TOOL." >> $OUTPUT
echo "For me, freedom means $FREEDOM." >> $OUTPUT
echo "I aim to build $BUILD and share it with the world." >> $OUTPUT

echo "--------------------------------------"
echo "Manifesto saved to $OUTPUT"
echo "--------------------------------------"

cat $OUTPUT
```

Output:

```
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$ ./script5_manifesto_generator.sh
======================================
 OPEN SOURCE MANIFESTO GENERATOR
======================================
1. Name an open-source tool: Linux
2. What does freedom mean to you? Exploring
3. What will you build and share? Strength and Kindness
--------------------------------------
--------------------------------------
Manifesto saved to manifesto_dixita_nema.txt
--------------------------------------
On 29 March 2026, I believe in open source.
I regularly use Linux.
For me, freedom means Exploring.
I aim to build Strength and Kindness and share it with the world.
dixita_nema@LAPTOP-L9HT61AT:/mnt/c/Users/Dixita Nema/OneDrive/Documents/OSS-Audit-24BAI10802$
```

Learning Outcomes

Through this project, I gained practical and theoretical knowledge of several important concepts in open-source software and system operations:

- **Linux command-line operations:** I learned how to navigate directories, manage files, and execute commands efficiently using the Linux terminal, which is essential for system-level tasks.
- **Bash scripting fundamentals:** I understood how to write and execute shell scripts using variables, loops, and conditional statements to automate tasks.
- **File system and permissions:** I explored how Linux manages file permissions, ownership, and directory structures, which is important for system security and organization.
- **Open-source licensing:** I gained knowledge about licenses such as GPL v2 and how they ensure software freedom and encourage collaboration.
- **Practical usage of Git:** I learned how Git works as a version control system and how it is used for tracking changes and collaborative development.

Challenges Faced

During the implementation of this project, I encountered several challenges that helped improve my technical and problem-solving skills:

- **Setting up WSL environment:** Initially, configuring Windows Subsystem for Linux was challenging due to permission issues and installation steps.
- **Understanding Linux file paths:** Converting Windows paths to Linux paths (like /mnt/c/...) was confusing at first but became clear with practice.
- **Debugging script errors:** Some scripts did not run correctly initially, and identifying syntax or logical errors required careful debugging.
- **Managing permissions:** Handling execution permissions using commands like chmod was a new concept that needed proper understanding.

These challenges enhanced my debugging ability and strengthened my understanding of Linux-based environments.

Conclusion

This project provided a comprehensive understanding of both Git and open-source software concepts. By implementing Bash scripts, I gained hands-on experience in working with Linux systems and automating tasks. The study of Git highlighted its importance in modern software development and collaborative environments. Overall, this project helped bridge the gap between theoretical knowledge and practical application.

References

- Official documentation of Git was used to understand its features and working principles.
- Linux manual pages (man commands) were referred to for understanding system commands and utilities.
- Resources from the Open Source Initiative were used to study open-source principles and licensing.
- GNU GPL v2 documentation was referred to understand licensing terms and conditions.